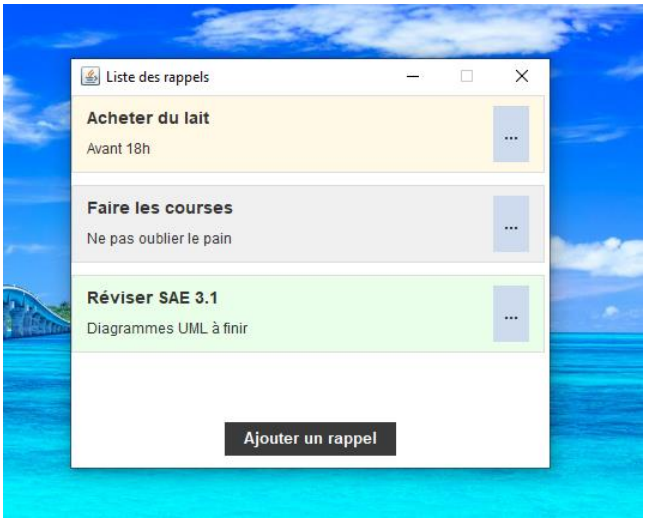
BUT Informatique – 2eme année
SAE 3.1 – Application “Papillon”

Développement d'une application de gestion de rappels

(Affichage permanent à l'écran)



Membres du groupe :

- Algassimou Pellel DIALLO (groupe4)
- Ayoub ANHDIRE (groupe4)
- Youness BOULALAM (groupe4)

Comptes GIT utilisé par personne pour ce projet :

- Algassimou Pellel DIALLO (groupe4) : - AlgaLaptop et Algassimou Pellel DIALLO
- Ayoub ANHDIRE (groupe4) : - Ayoub ANHDIRE et Anhdire Ayoub
- Youness BOULALAM (groupe4) : -Youness BOULALAM et ynssb

Réalisé en Java – Interface Swing – Architecture MVC

Dépôt Gitea [https://grond.iut-fbleau.fr/dialloa/SAE31_2025.git]

Professeur : M. Luc HERNANDEZ

RAPPORT SAE31_2025

Date de rendu : 26 octobre 2025

Table des matières

1. **Introduction**
2. **Répartition des tâches**
3. **Fonctionnalités principales**
    a. 3.1 Affichage de la liste des rappels triés par rang (Diallo)
    b. 3.2 Création d'un nouveau rappel (Andhire)
    c. 3.3 Consultation d'un rappel en détail (Diallo)
    d. 3.4 Modification d'un rappel existant (Andhire)
    e. 3.5 Suppression d'un rappel (Boulalam)
    f. 3.6 Changement d'ordre des rappels (Diallo & Andhire)
4. **Organisation de la base de données *(à compléter par Youness)***
5. **Makefile Du Projet**
6. **Conclusion**

1. Introduction :

Dans le cadre de la SAE 3.1, il nous a été demandé de concevoir un petit logiciel appelé Papillon, dont le but est d'afficher en permanence une liste de rappels à l'écran, un peu comme des post-it collés sur le bord du moniteur. Le but est que l'utilisateur garde toujours un œil sur ses rappels sans que cela gêne l'utilisation des autres applications. Ce projet a été réalisé en trinôme, et nous avons développé l'application entièrement en Java, en respectant les bonnes pratiques vues en cours : architecture MVC, création de makefile, et une interface graphique codée sans bibliothèque externe (seulement Swing et AWT). Les rappels peuvent contenir un titre, un contenu optionnel, un thème (couleur), et un rang pour permettre le tri. L'utilisateur doit pouvoir créer, modifier, supprimer, et consulter ses rappels facilement. Les données sont stockées dans une base de données distante (MariaDB), afin qu'elles soient conservées même après la fermeture de l'application. Un soin particulier a été apporté à l'ergonomie : l'interface est pensée pour être simple, claire, et agréable à utiliser.

2. Répartitions des Taches :

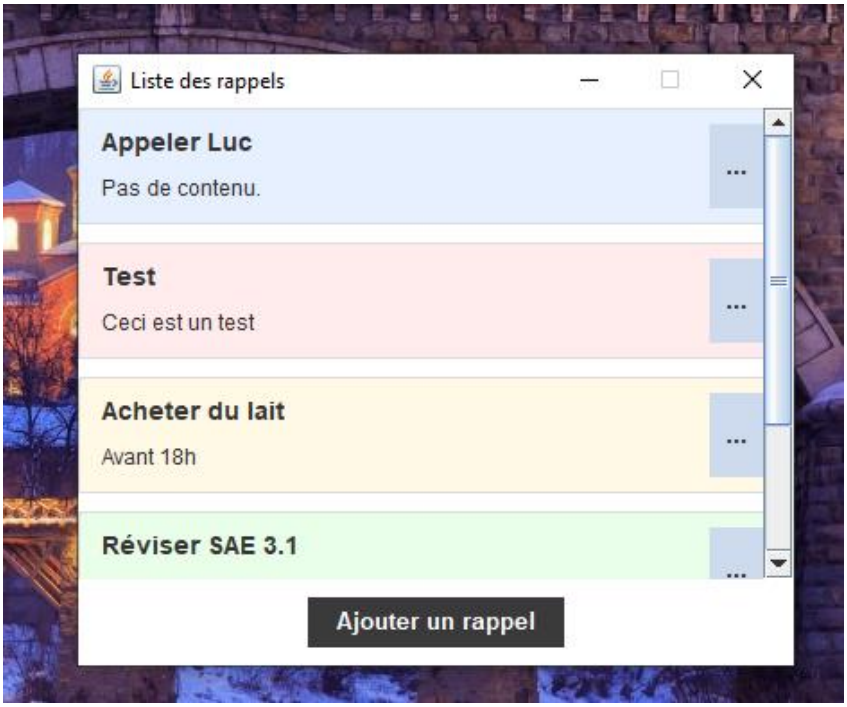
Développeur	Tache
Algassimou Pellel DIALLO	• Développement de toutes les classes du modèle : ModelRappel.java, DBManager.java, ModelRappelDAO.java • Définition de l'architecture MVC globale et création de l'arborescence de fichiers • Codage de l'affichage de la liste des rappels • Codage de l'affichage détaillé de chaque rappel • Développement de la logique de déplacement des rappels (changer l'ordre d'affichage selon l'importance) • Création et gestion de la classe ErrorManager (messages d'erreur, d'alerte et de confirmation) • Codage du contrôleur. • Création du listener de fermeture (ListenerFermeture) pour assurer une fermeture propre de l'application + Connection • Création du makefile • JAVADOC (En équipe)

Ayoub ANHDIRE	- Ajout de méthodes dans les classes du modèle dans ModelRappelDAO.java tel qu'ajouter un Rappel, modifier un rappel ou prendre un rappel en particulier - Codage de la vue de la création d'un Rappel - Codage de la vue de la modification d'un Rappel - Faire les liens entre les différents panels (avec le CardLayout) - La JAVADOC (en équipe) - Ajout de boutons pour augmenter ou descendre le rang d'un Rappel dans la Vue Détail Panel - Gérer les données écrites dans les vues Création et Modification pour pas que l'utilisateur entre des données nulles dans la BD pour après les envoyer au contrôleur. - Gérer que la fenêtre soit toujours au 1er plan. - Gérer le fait que quand l'utilisateur choisit un thème il soit appliqué directement à l'écran.
Youness BOULALAM	

3. Fonctionnalités principales :

Notre programme contient plusieurs fonctionnalités comme :

3.1 Affichage de la liste des rappels trier par rang (Implémenté par Algassimou Pellel DIALLO)



Cette fonctionnalité permet d'afficher à l'utilisateur la liste complète de ses rappels, classés automatiquement selon un ordre de priorité défini en interne via un champ appelé rang. Ce tri garantit que les rappels les plus importants (rang faible) apparaissent en haut de l'écran.

J'ai conçu et mis en place le système d'affichage des rappels dans l'interface principale. Initialement, j'avais utilisé une classe ComparateurRang implémentant l'interface Comparator, pour trier les rappels côté Vue. Mais après réflexion, j'ai choisi une méthode plus propre et plus efficace : le tri est désormais délégué directement à la base de données via la requête SQL. Ainsi, les rappels sont récupérés déjà triés par rang, ce qui allège la vue et clarifie la séparation des responsabilités dans le modèle MVC.

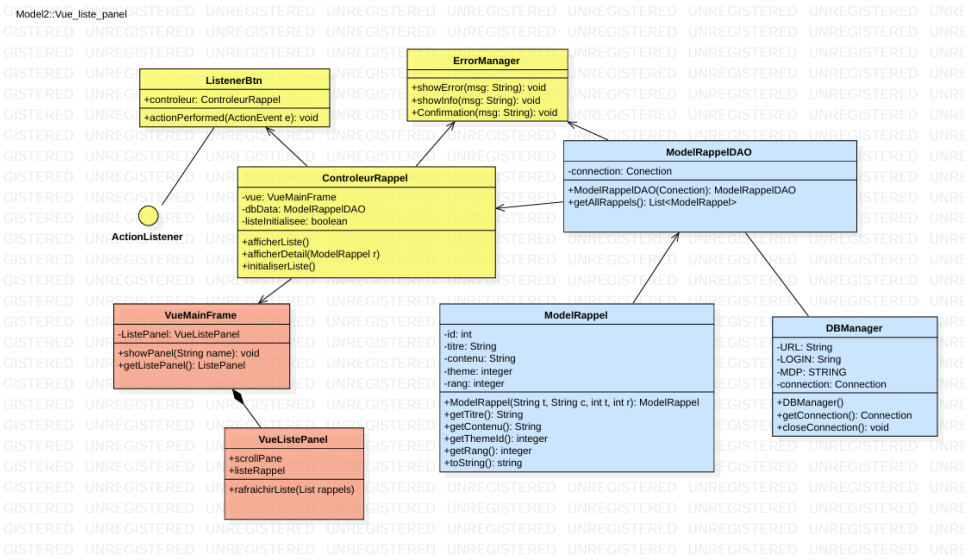
La principale difficulté a été de gérer les boutons individuels permettant d'accéder à la vue de détail (les boutons "**...**" présents sur chaque carte de rappel). Chaque rappel affiché doit posséder son propre bouton relié à un ListenerBtn, avec l'ID du rappel correspondant. Mettre en place proprement ces boutons dynamiques sans erreur de liaison ni duplication a demandé beaucoup de rigueur dans la gestion des listeners.

J'ai remarqué un ralentissement lors du retour depuis la vue "détail" vers la liste. En effet, chaque retour entraînait un rechargement complet de la base et la réattribution des listeners à chaque carte (bloc de rappel). Pour éviter cela, j'ai mis en place un système de cache simple, basé sur une variable listeInitialisee. La liste complète n'est rechargée que lorsqu'il y a une vraie modification dans la base (ajout, suppression, modification). De plus, à l'ajout d'un rappel, seul ce

nouveau rappel est inséré dans la liste, sans devoir tout recharger. Cela rend l'interface plus rapide et réactive.

Pour aller plus loin, il serait possible d'ajouter une pagination ou un filtrage dynamique si le nombre de rappels devient important. Mais bon le temps ne m'a pas permis de le faire.

Diagramme de classe UML :



3.2 Création d'un nouveau rappel (implémenter par Ayoub ANHDIRE)



Cette fonctionnalité permet entre autres de donner la possibilité à l'utilisateur de créer un *nouveau Rappel*.

Cette vue contient plusieurs composants graphiques tels que des champs et des zones de textes, des boutons à cocher et des boutons. Expliquons chacun de ces composants : le premier composant est un champ de Texte : il permettra à l'utilisateur de pouvoir donner un titre à son rappel : on a décidé d'utiliser un champ de Texte par rapport à une zone de texte car un titre fait au **maximum** 50 caractères ce qui correspond à peu près à une ligne environ d'où l'utilisation d'un champ de Texte. Si par malheur, l'utilisateur écrit un titre supérieur à 50 caractères un message d'erreur lui sera affiché lorsqu'il voudra appuyer sur le bouton « Ajouter ce rappel ».

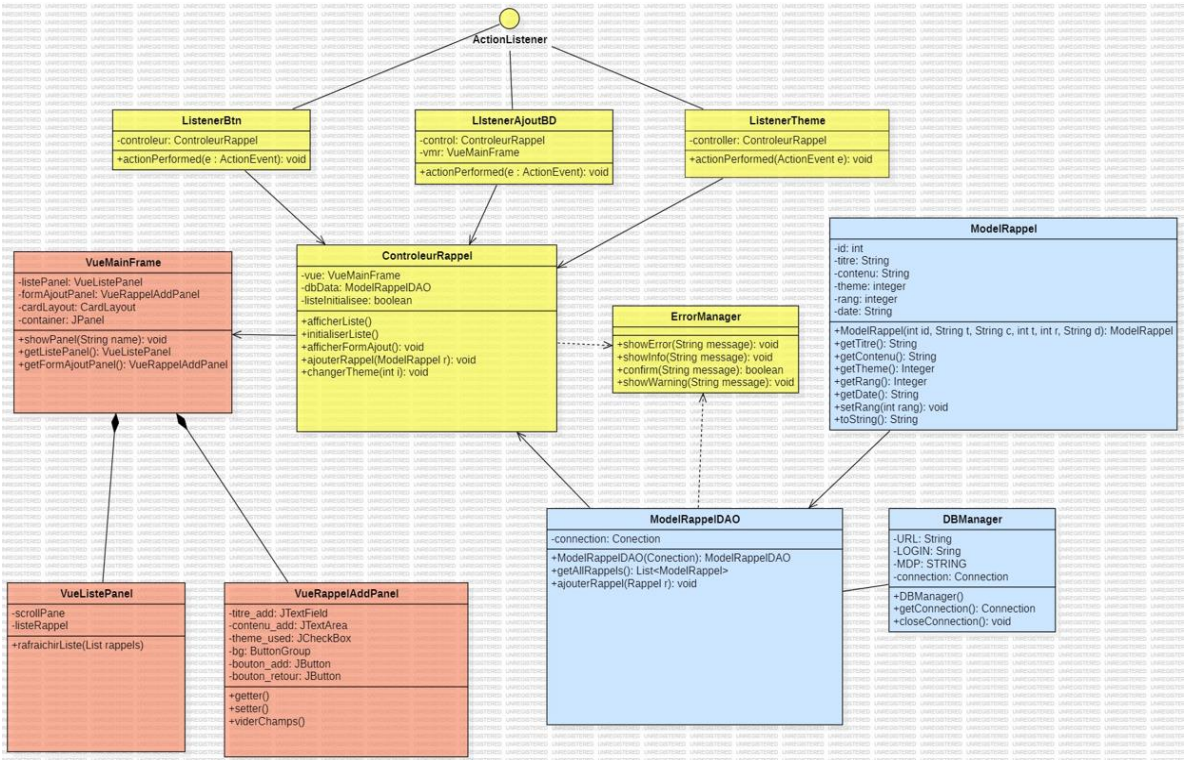
Le deuxième composant graphique est une zone de Texte où l'utilisateur a le choix ou non de pouvoir mettre un contenu, comme rappelé plus haut le contenu est optionnel : tout dépend de la volonté de l'utilisateur. Tout comme le titre, le contenu a une taille à ne pas dépasser, cette dernière est de 200 caractères d'où l'utilisation d'un JTextArea : celui-ci a été codé de façon que quand l'utilisateur est en fin de ligne, il soit redirigé sur une nouvelle ligne et s'il veut accéder à ce qu'il a écrit plus haut, il pourra se servir du JScrollPane s'il en désire.

Le troisième composant graphique sont entre autres des boutons à cocher qui représentent le thème d'un Rappel, comme indiqué dans le sujet de cette SAé, nous avons décidé d'en proposer 5 à l'utilisateur qui sont en autres des couleurs distinctes comme le rouge, le bleu, le vert, le jaune et le gris. Ils sont représentés sous forme de BoutonRadio, tout cela a été possible avec la classe **JCheckBox**. Pour améliorer l'User Expérience, nous avons pensé qu'il était plus préférable à ce que l'utilisateur voit ce que donne le thème pour pouvoir ensuite choisir ce qu'il préfère avant de valider l'ajout du Rappel. C'est pour ça que nous avons décidé de définir une classe **ListenerTheme** que lorsqu'un bouton est cliqué le thème est appliqué directement. Par logique, nous avons pensé que l'utilisateur pouvait choisir un seul et unique Thème, mais en autres les boutons à cocher peuvent être sélectionnés même si les autres sont déjà sélectionnés, c'est pour ça que nous avons utilisé la classe **ButtonGroup** contenant plusieurs méthodes mais celle qui nous intéressait dans notre cas était la méthode **add**() qui permet de sélectionner un seul et unique bouton à la fois.

Les deux derniers composants graphiques sont deux boutons d'où par leur nom on peut déduire qu'un permet de revenir sur la liste des Rappels et que l'autre permet d'ajouter le Rappel. Concentrons-nous sur le 1er bouton « Ajouter ce Rappel », lorsque l'utilisateur appuie sur ce bouton, nous avons une classe qui permet de gérer les données entrées par l'utilisateur : **ListenerAjoutBD**. Ce dernier permet entre autres de vérifier ce que l'utilisateur écrit dans la vue. Si par malheur, l'utilisateur ne choisit aucun thème et/ou aucun Titre : une fenêtre lui sera affiché comme quoi qu'il n'ait complété que la moitié ou aucune des informations obligatoires. C'est grâce à la classe **ErrorManager** que l'on réalise cela notamment avec plusieurs méthodes déclarés selon différentes situations. On utilise des fenêtres déjà préconstruites : les **JOptionPane**. Si l'utilisateur entre tous les champs correspondants et que les contraintes sont respectées, alors on créer un Objet **ModelRappel** avec les données récupérées qu'on envoie à la classe **ContrôleurRappel** et puis qu'il envoie à la classe **ModelRappelDAO** qui a une méthode **ajouterRappel (ModelRappel r)** qui permet d'ajouter le rappel dans la base de données. Après cela, l'utilisateur est redirigé automatiquement vers la liste des Rappels qui sera rafraîchi et il pourra voir le rappel tout fraichement ajouté.

Le deuxième bouton « Retour» permet de revenir à la liste des Rappels d'où son nom. Lorsqu'il est cliqué ; il est redirigé vers la liste des Rappels. Cela est réalisé grâce aux CardLayout et au Listener sur ce bouton. Lorsque ce bouton est cliqué, on récupère le contenu et une action est déclenché : dans la classe **ListenerBtn**, si l'action correspondante est « retour », alors on appelle une méthode du contrôleur qui permet d'afficher la liste des Rappels, qui elle-même appelle une méthode dans **VueMainFrame,** la méthode show Panel qui permet grâce à la méthode show () de CardLayout d'afficher un Panel en particulier**.**

Diagramme de classe UML :



3.3 Consultation d'un rappel en détail(Algassimou DIALLO)

Permettre à l'utilisateur de consulter toutes les informations d'un rappel (titre, contenu, date, thème...) dans une interface, après avoir cliqué sur un rappel de la liste principale (plus précisément sur le bouton "...").

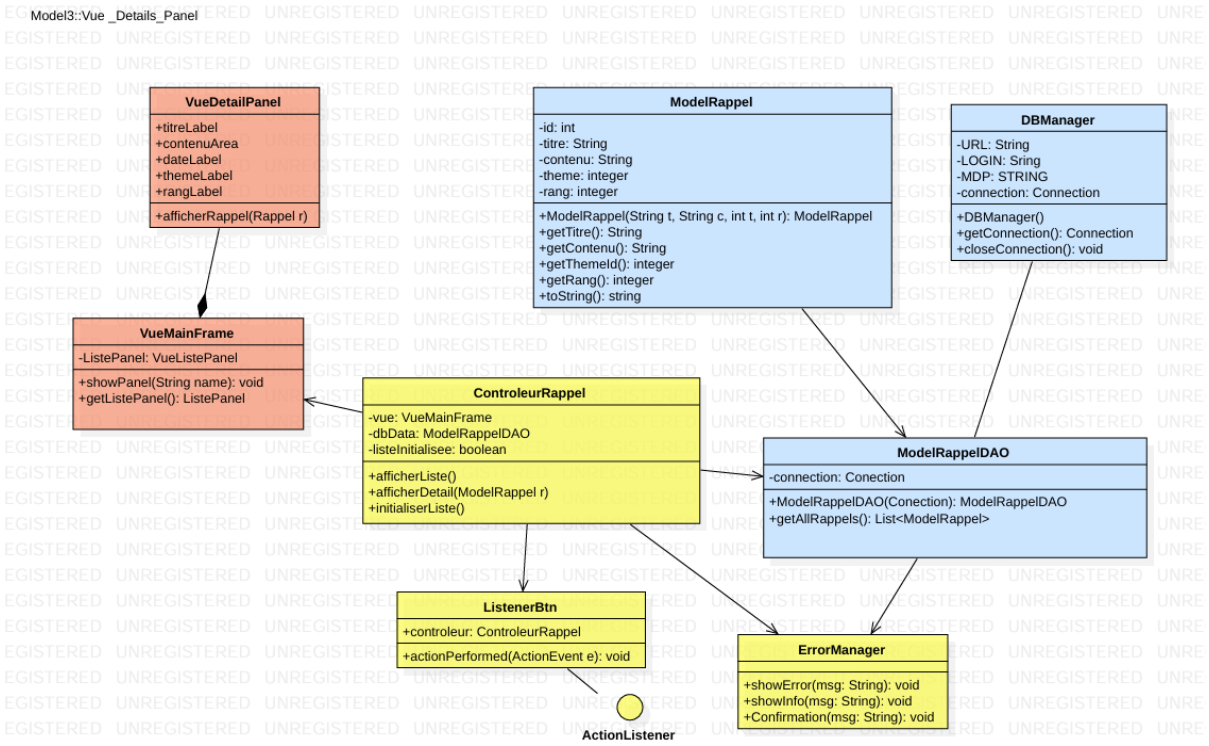


Pour cela, j'ai conçu la classe **VueDetailPanel** qui représente la vue dédiée à l'affichage du détail d'un rappel. Cette vue a été intégrée dans l'interface principale **(VueMainFrame)** à travers un système de **CardLayout**, ce qui permet de basculer dynamiquement entre la liste des rappels et le détail d'un rappel sélectionné. L'appel de cette vue est géré via le contrôleur, en utilisant une

méthode **afficherDetail(ModelRappel)** que j'ai mise en place pour charger les données du rappel sélectionné dans les composants graphiques de la vue détail et ensuite le contrôleur affiche le panel détail. Ces composants sont principalement des **JLabel** et **JTextArea**, qui affichent respectivement le titre, le thème, la date, ainsi que le contenu du rappel. J'ai également intégré un bouton de retour permettant à l'utilisateur de revenir facilement à la vue de la liste sans perdre de données ni casser l'expérience utilisateur. Et en dernier j'ai rajouté les bouton Modifier, supprimer, Retour et Ayoub a ajouté Monter qui fais remonter un rappel dans la liste en diminuant sont rang, Descendre fais l'inverse.

La principale difficulté rencontrée concernait la gestion des interactions utilisateur dans la vue détail, notamment les boutons de modification, de suppression et de retour. Au début, ces boutons posaient un problème important : à chaque navigation entre la liste des rappels et la vue détaillée d'un rappel, les **ActionListener** s'ajoutaient les uns aux autres au lieu d'être remplacés. Ce comportement provoquait l'exécution multiple d'une même action, par exemple plusieurs boîtes de dialogue de confirmation qui s'ouvraient simultanément, rendant l'application confuse.
Pour résoudre ce problème, j'ai mis en place une méthode spécifique appelée **resetListeners()**. Cette méthode supprime tous les anciens écouteurs d'un bouton avant d'en ajouter de nouveaux, garantissant ainsi qu'un seul **ActionListener** est actif à la fois. Grâce à cette approche, chaque bouton conserve un comportement unique et prévisible, même après plusieurs allers-retours entre les différentes vues.
J'ai également rencontré des difficultés pour identifier précisément le rappel associé à chaque bouton de la liste. Il fallait que, lors d'un clic sur un bouton, le programme sache exactement de quel rappel il s'agissait afin d'en afficher les informations. Après plusieurs recherches dans la documentation Swing, j'ai trouvé une solution efficace en utilisant les méthodes **putClientProperty ()** et **getClientProperty ()**. Ces méthodes permettent d'associer directement un objet **ModelRappel** à un composant graphique. Ainsi, chaque bouton de la liste est désormais lié à son rappel correspondant, ce qui facilite grandement la récupération et l'affichage des données dans la vue détail. Comme chaque bouton de la liste des rappels est déjà lié à un objet **ModelRappel** via la méthode **putClientProperty()**, toutes les informations nécessaires sont disponibles directement dans l'interface. Cela permet de transmettre le rappel concerné au contrôleur sans faire de nouvel appel au DAO. Cette optimisation réduit la charge sur la base de données et rend l'affichage de la vue détail quasi instantané, même après plusieurs interactions successives, en conservant en mémoire les rappels déjà affichés dans la liste.

Diagramme de classe UML:



3.4 Modification d'un rappel existant (implémenté par Ayoub ANHDIRE)



Cette fonctionnalité permet entre autres de modifier un Rappel déjà existant : comme vous pouvez le constater elle ressemble énormément à la vue de la création d'un Rappel mais il y a quelques différences. On peut accéder à cette vue en appuyant sur le bouton Modifier sur la VueDetailPanel.

Nous avons toujours les mêmes composants graphiques, un **JTextField** pour le titre du Rappel, un **JTextArea** pour le contenu du Rappel, des boutons à cocher (**JCheckBox**) avec les 5 thèmes et les 2 boutons (**JButton**) mais leur fonction désormais change.

Le premier composant de cette vue est un **JTextField**. Contrairement à la vue de la création de Rappel, celui-ci est déjà préremplie avec le titre du Rappel à modifier : cela a été fait dans le but que l'utilisateur se souvienne du nom de titre qu'il ait mit et qu'il puisse ainsi le changer à sa façon. Les mêmes contraintes que pour la création sont respectés ici : un titre vide ou trop long sera indiqué à l'utilisateur via une boite de dialogue graphique avec un message d'avertissement. C'est la classe **ErrorManager** qui gère cela.

La question qu'on se pose c'est comment ses données sont chargés, à vrai dire dans la **VueRappelModifPanel**, nous avons une méthode **afficherRappelModif (ModelRappel r)**, qui permet d'initialiser les champs et zones de Texte selon le Rappel en question, et avec les méthodes de la classe **ModelRappel** on peut mettre les données dans les zones réservés.

Le deuxième composant graphique est une zone de texte : un **JTextArea** pour le contenu du rappel. Il respecte les mêmes contraintes que celle définies pour une création de Rappel. De plus, il respecte les mêmes fonctions que celle définies pour la création d'un Rappel : passe automatiquement à la ligne quand il arrive à la fin de cette dernière et un **JScrollPane** qui lui permet de naviguer dans cette zone de texte. Le contenu peut être **nulle**, mais ne doit pas dépasser plus de 200 caractères. Il est initialisé de la même façon que pour le **JTextField** défini juste en haut.

Le troisième composant graphique sont 5 boutons à cocher (**JCheckBox**) qui permettent de choisir un thème comme pour la création du rappel. Il ne peut choisir qu'un seul thème(**ButtonGroup**) et peux changer sa couleur s'il en éprouve le besoin.

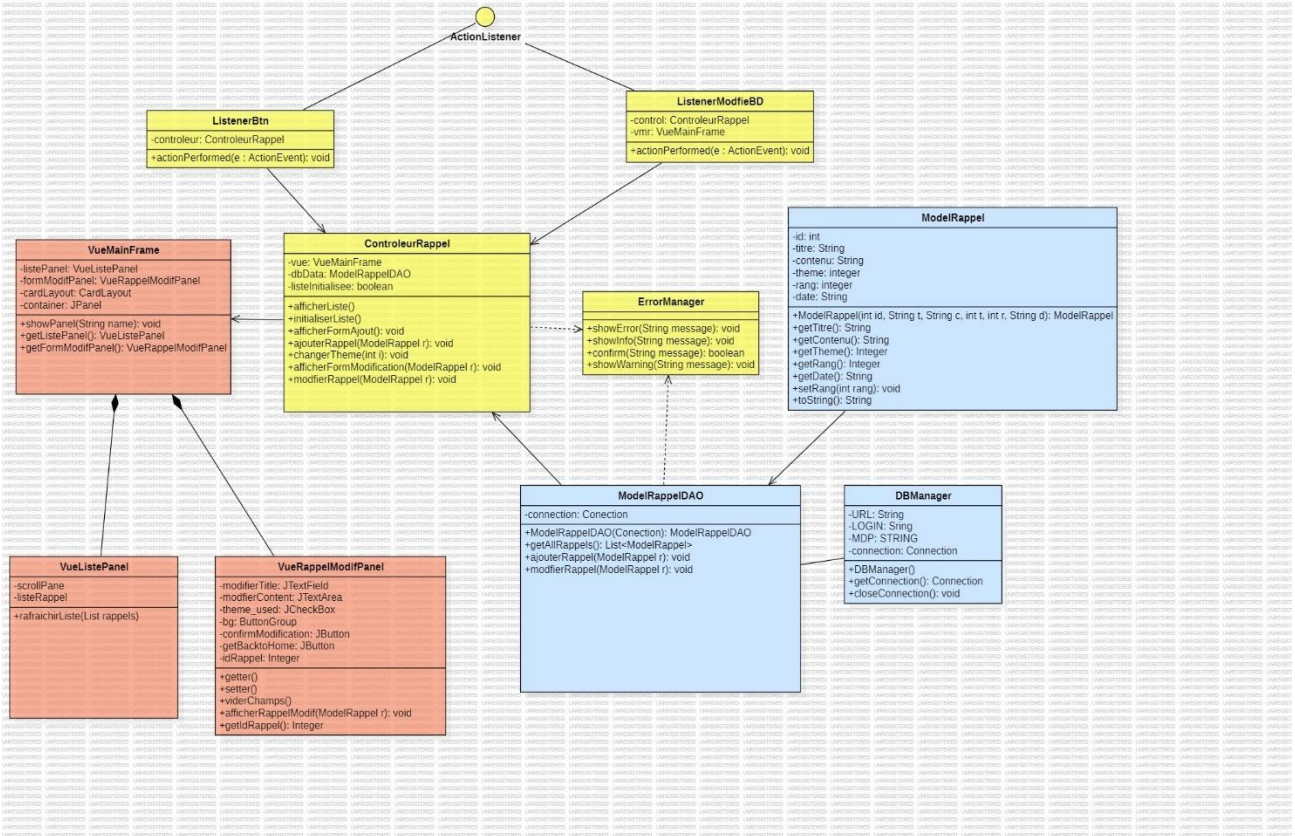
Les deux derniers composants graphiques sont deux boutons « Conserver les modifications » et « Revenir à l'accueil ». Expliquons leurs utilités et l'action qui se passe lorsque ces derniers sont cliqués.

Pour le bouton « Revenir à l'accueil » c'est simple et ça ressemble énormément au bouton « Retour » de la vue de la création d'un Rappel. Lorsque ce dernier est cliqué, la classe **ListenerBtn** permet de définir une action. Si ce dernier est cliqué, alors on appelle la méthode **afficherListe ()** du contrôleur **ControleurRappel** qui appelle la vue **VueMainFrame** et plus précisément la méthode **showPanel (String name).** Cette méthode permet d'afficher un cardLayout en particulier selon le nom qu'on a donnée aux panneaux lorsqu'on

les a ajoutés au conteneur. Ducoup en faisant showPanel (« Liste ») par exemple , le conteneur montrera la vue **VueListePanel**.

Pour le bouton « Confirmer les modifications » , il y a plusieurs cas possibles : soit l'utilisateur ne rentre aucune information et ducoup il est prévenu via la classe **ErrorManager** qu'il faut rentrer des informations : ou sinon s'il a parfaitement rempli tout les champs obligatoires alors un objet **ModelRappel** sera créé : c'est là que ça se complique : au début je ne savais pas comment créer cet objet car en réalité nous réalisons une requête SQL « UPDATE » sur la base de données mais pour faire cette requête , je dois la réaliser uniquement sur le rappel en question et non sur tout les rappels qu'il y a dans la base de données. Après ça , j'ai su qu'il fallait récupérer l'ID correspondant au rappel pour appliquer la clause WHERE dessus, et c'est pour ca que dans la classe **VueRappelModifPanel** nous avons un attribut *idRappel* que l'on récupérer et que l'on insère dans le constructeur du nouvel objet **ModelRappel** et cet objet on l'envoie dans la méthode **modifierRappel(ModelRappel r)** du contrôleur qui lui permet d'envoyer à la classe ModelRappelDAO le rappel correspondant qui lui exécutera la requête SQL « UPDATE » correspondante. L'utilisateur sera prévenu si le rappel a bien été modifié ou non.

Diagramme de classe UML :



3.5 Suppression d'un rappel (implémenté par Youness BOULALAM)

3.6 Choix de d'ordre sur le rappel (implémenter par Algassimou DIALLO (BackEnd) et Ayoub ANHDIRE (Frontend)).

Frontend :

Pour permettre à l'utilisateur de modifier le rang d'un Rappel, de pouvoir le manipuler comme il le souhaite j'ai ajouté dans la **VueDetailPanel** deux boutons « Monter » et « Descendre » que lorsqu'ils sont cliqués on peut descendre ou augmenter le rang d'un Rappel. Comment cela marche-t-il ? Alors pour avoir réalisé cela : on ajoute des écouteurs sur les deux boutons correspondants. Les actions définis par ces boutons lorsqu'ils sont cliqués sont dans la classe **ListenerBtn**. Si un bouton est cliqué, on appelle la méthode **deplacerRappel(int id, boolean versLeHaut)** de **ControleurRappel** qui en permet d'échanger des rangs.

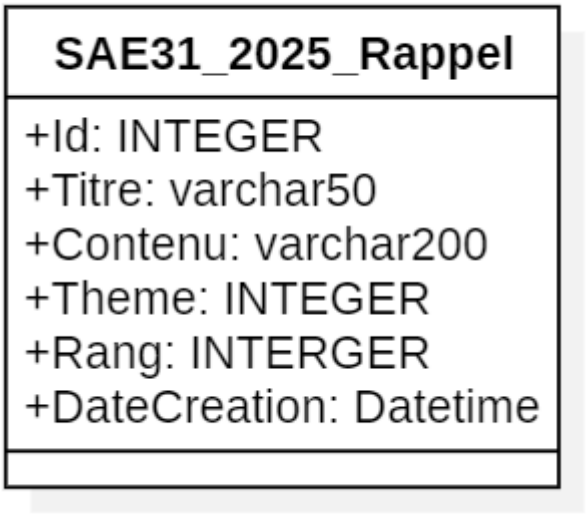
Backend :

Pour permettre à l'utilisateur de réorganiser ses rappels selon ses préférences, j'ai aidé à la mise en place coté backend d'une logique de déplacement (vers le haut ou vers le bas). L'idée était de traiter le champ rang comme un critère d'ordre abstrait, invisible pour l'utilisateur, mais nécessaire pour l'affichage. Ainsi, lorsque l'utilisateur souhaite déplacer un rappel, le programme doit simplement échanger son rang avec celui du rappel voisin (précédent ou suivant). La méthode **deplacerRappel(int id, boolean versHaut)** du contrôleur centralise cette logique. Elle commence par récupérer le rappel à déplacer via le DAO, puis identifie son voisin immédiat dans la direction souhaitée. Si ce voisin existe, un échange des

rangs est effectué, suivi d'une mise à jour des deux rappels dans la base de données. Cette approche évite toute reconstruction complète de la liste, ce qui la rend simple et peu coûteuse. Pour garantir la cohérence des rangs après des suppressions, j'ai également ajouté une méthode **reorganiserRangs()** dans le DAO. Elle est appelée automatiquement après chaque suppression de rappel, afin de recompacter les rangs et garantir leur continuité (0, 1, 2, 3, etc.). Cela permet de maintenir une base de données propre et un affichage sans trous dans l'ordre.

4. Organisation de la Base de Donne

BASE DE DONNE à faire Youness tu dois expliquer la bd et toutes les contraintes pourquoi on les a ajoutés



5. MAKEFILE DU PROJET

Une composante essentielle du projet a été la création d'un Makefile capable d'automatiser toutes les étapes clés : compilation, exécution, génération des JAR et documentation. Grâce à ce fichier, tout le processus de compilation est centralisé et simplifié, évitant ainsi les erreurs manuelles et les commandes répétitives. Le Makefile permet non seulement de générer un JAR exécutable classique **(Papillon.jar)**, mais aussi un JAR autonome (**Papillon-FAT.jar**), qui embarque directement le driver **MariaDB**. Ce second JAR est particulièrement pratique pour la livraison du projet, car il fonctionne sans dépendance externe.

Chaque composant du projet (modèle, vues, contrôleur et aussi les listeners) est compilé avec ses propres dépendances. Cela permet de ne recompiler que ce qui a changé, tout en respectant l'ordre logique des modules. Cependant, j'ai rencontré des difficultés particulières avec la compilation du contrôleur et des listeners. En effet, ces fichiers dépendent du contrôleur et le contrôleur dépend d'eux, ce qui créait des

dépendances circulaires. En temps normal, on pourrait utiliser la variable automatique $^ pour reprendre directement toutes les dépendances listées au-dessus. Cependant, dans cette règle, cela posait problème. En effet, $^ aurait inclus les fichiers .class comme arguments à javac, alors que javac attend uniquement des fichiers .java à compiler. Pour éviter cette erreur, j'ai donc explicitement listé tous les fichiers .java à compiler, juste après le javac, même si cela semble redondant. Cette solution garantit que seules les sources Java sont compilées, tout en conservant les fichiers .class dans les dépendances pour forcer leur compilation préalable. L'autre difficulté rencontrée lors de l'écriture du make a été liée à la portabilité entre systèmes d'exploitation. Chaque membre de l'équipe travaillait sur un système différent **(Windows, Linux…)**, ce qui posait un problème avec la gestion du classpath, notamment à cause du séparateur : sur Linux et ; sur Windows. Pour résoudre cela, j'ai intégré dans le Makefile une condition **ifeq ($(OS),Windows_NT)** afin d'adapter dynamiquement le séparateur **$(SEP)** en fonction du système détecté. Grâce à cette solution, le Makefile fonctionne de manière portable, sans qu'on ait à le modifier à chaque changement d'environnement.

Voici les commandes disponibles dans le Makefile :

make	# Compile tout le projet (équivalent à make all)
make runnotjar	# Exécute le projet sans passer par un JAR (classes compilées dans /build)
make jar	# Crée un JAR exécutable (Papillon.jar)
make fatjar	# Crée un JAR autonome (Papillon-FAT.jar) incluant le driver MariaDB
make run	# Lance l'application depuis Papillon.jar
make runfatjar	# Lance l'application depuis Papillon-FAT.jar
make doc	# Génère la documentation Java dans le dossier /docjava
make clean	# Supprime les fichiers compilés (.class), les JAR et la documentation

6. Conclusion

Algassimou Pellel DIALLO : Cette SAE m'a permis de prendre en main l'ensemble du cycle de développement d'une application Java structurée en MVC, tout en respectant une méthode de travail en équipe. J'ai pu développer plusieurs fonctionnalités clés, comme l'affichage dynamique des rappels, leur consultation en détail, la gestion fluide des événements utilisateur, et la logique de déplacement des éléments dans la base. J'ai également appris à mieux séparer les responsabilités entre le contrôleur, les vues et le modèle, ce qui a grandement amélioré ma compréhension

de l'architecture MVC. L'un des apports majeurs de cette SAE a été la découverte et la mise en œuvre d'un Makefile complet. Avant ce projet, je ne savais même pas comment créer un Makefile. J'ai appris à automatiser toutes les étapes : compilation, exécution, génération de la documentation, création de JAR classiques et JAR "autonomes". Cette partie m'a également appris à gérer les problèmes de portabilité entre systèmes (Linux/Windows), en utilisant des variables comme $(SEP) pour gérer correctement le classpath. Enfin, travailler avec une approche agile et collaborative m'a permis de mieux comprendre l'importance de la communication, de l'adaptation et de la clarté dans la répartition des tâches. Cela m'a donné de l'autonomie tout en me poussant à documenter mon code, à penser aux autres membres de l'équipe et à rendre mes choix de conception compréhensibles.

Ayoub ANHDIRE : Pour conclure, j'ai bien aimé ce projet en général, j'ai pris du plaisir à coder en JAVA d'autant plus que j'affectionne la programmation orientée objet. Ce projet m'a permis d'augmenter mes compétences techniques en JAVA, plus précisément dans l'interface graphique avec l'usage des Bibliothèque SWING et AWT mais pas que j'ai pu aussi revoir comment gérer une base de données et communiquer avec elle via des requêtes SQL. J'ai pu aussi développer mes qualités de communication avec mes camarades : chacun a joué un rôle ou il sait qu'il va perfectionner et la communication a été un enjeu majeur dans cette SAé car lorsque quelqu'un était bloqué, il faisait signe et ne restait pas tout seul sans avancer dans sa tâche. En conclusion, ce projet a été pour moi une expérience enrichissante d'où je tirerai certainement des profits.

Youness BOULALAM :